

CSC2062 AIMA – Assignment 2

[Ammar Jileidan]

[40450780]

[17/04/2026]

Introduction

In this project, logistic regression classification, K nearest neighbour and random forest to classify images.

Section 1

To identify the most discriminative features for the models, The Pearson correlation coefficient between each feature and its category as living=1 and non-living=0 was calculated and ranked based on the highest absolute correlation values. As shown in the feguares below, the four feautres with highest absolute corelation are : eyes, rows_with_1, maxrow with correlation values -0.39, -0.35, -0.34 and -0.29. The negative values indicating that the living objects tend to have lower values for these features compared to the non-living objects.

Feautre/living correlation:	
rows_with_2	0.2176
hollowness	0.2113
connected_areas	0.2107
cols_with_2	0.1960
cols_with_1	0.1382
aspect_ratio	0.0685
width	-0.0000
rows_with_3p	-0.0000
height	-0.0269
img_bdensity	-0.1485
cols_with_3p	-0.1961
nr_pix	-0.2581
maxcol	-0.2915
maxrow	-0.3445
rows_with_1	-0.3479
eyes	-0.3900

Top 4 correlated features(they will be used in the models):

eyes
rows_with_1
maxrow
maxcol

Section 1.1

Using stratified sampling, the 112 images were split into a training set of 88 items and a test set of 24 items, 3 images were randomly selected from each of the 8 classes for the test set, ensuring a proportional representation of all classes and preventing any class from over or under-representation in the evaluation.

A logistic model were fitted on the standardised data to predict the probability of an item belonging to the living category with a decision threshold 0.5.

```

+++++
logistic classification metrics
+++++
                        Confusion Matrix
          Pred Living  Pred Nonliving
Actual Living      9           3
Actual Nonliving   0           12
-----
False Positive Rate : 0.0000
-----
Accuracy            : 0.8750
-----
Precision           : 1.0000
-----
Recall/TPR          : 0.7500
-----
F1-Score            : 0.8571
-----

```

The model with a decision threshold 0.5 scored an accuracy of 0.875 correctly classifying 21 out of 24 which can indicate a good performance considering the limited use of features and the size of test items. While the precision of the model is 1 and the false positive rate is 0 indicating that the model made no false positive prediction, the true positive rate/ recall is 0.75 mispredicting 3 living items. This suggests that the model is conservative in predicting the items belong to the living category however, the F1 score of 0.857 reflects a good balance between precision and recall and indicates the model performed well. Overall, the model performed well generally but it prioritized not to predict a non-living as a living creating a deficiency in predicting all living objects.

Section 1.2

To achieve a 7-fold Cross validation, a similar sampling approach was followed as in task 1.1, for each fold 2 objects from each class were randomly selected resulting in a total of 16 items in each fold using sklearn's method `StratifiedKFold()`. The data was standardized within each fold independently to avoid data leakage. Using logistic regression model, the same decision threshold of 0.5 and exactly the same features as in the previous model in task 1.2. The model's average metrics across all 7 folds are shown below.

```

=====
Task 1.2: using 7-fold Cross validation
=====

+++++
Average metrics over the 7 folds
+++++
Accuracy : 0.8571
-----
TPR/Recall: 0.8214
-----
FPR      : 0.1071
-----
Precision : 0.8897
-----
F1       : 0.8479
-----
=====

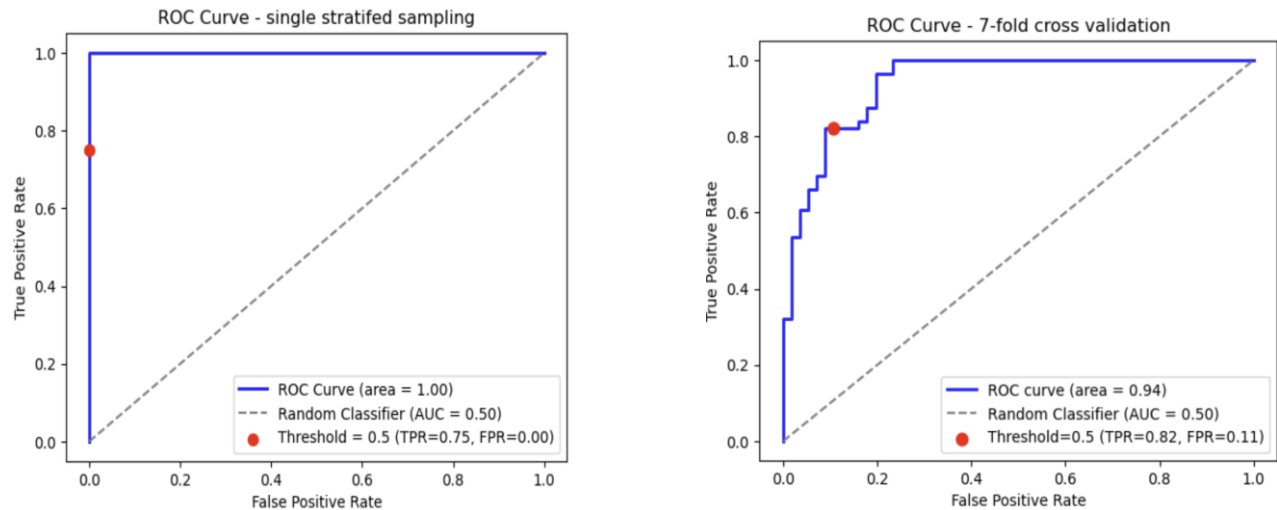
```

Using Cross Validation resulted in slight changes across the model's metrics. The perfect FPR of 0.0 and precision of 1.0 previously, changed to 0.108 and 0.89 respectively, making a more realistic estimation of the model's performance. The Accuracy slightly decreased from 0.875 to 0.857, while the Recall improved from 0.7500 to 0.8214, suggesting the model is less conservative in predicting the positive class when evaluated more broadly across all 112 samples. The F1 score remained relatively stable, moving from 0.857 to 0.848, confirming that the overall balance between precision and recall is well maintained. Overall, the CV results confirm that the model generalises well to unseen data, and provide a more trustworthy evaluation than a single stratified sampling.

Section 1.3

Two ROC curves were plotted. One for the single stratified sampling model and one for the 7-fold CV model. The red dots in each curve represent the performance at decision threshold of 0.5.

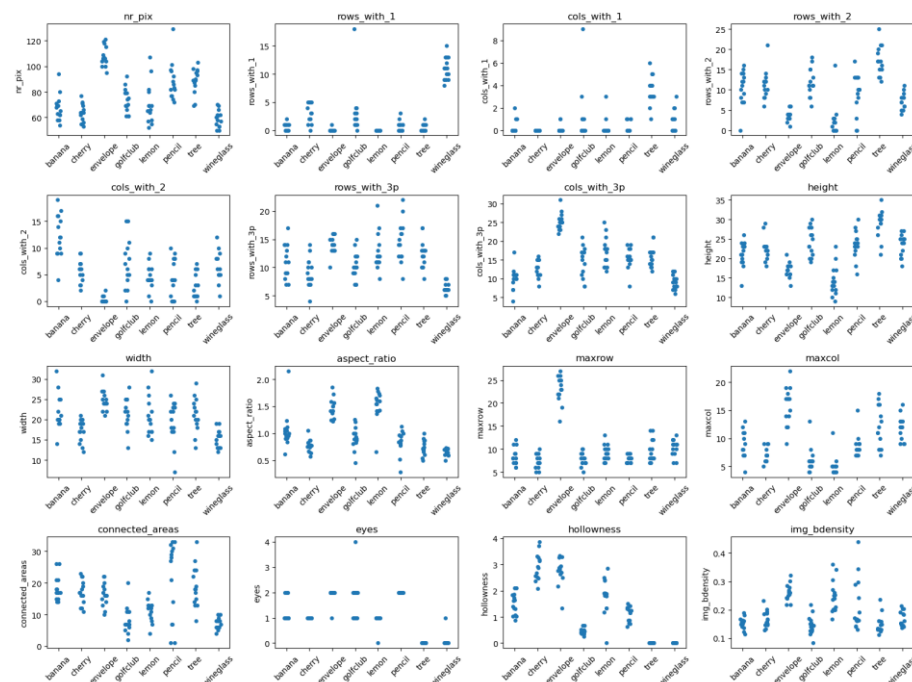
The first curve showed a perfect AUC of 1.0 which is optimistic as it was mentioned previously due to the limitation of test data while the CV's showed a more realistic AUC of 0.94 benefiting from being trained and



tested use the all 112 images. Both curves showed a nearly similar good performance at decision threshold of 0.5 and were both sitting well above the random classifier confirming their strong classifying power.

Section 2

In Task 1, the classification was between the living and non-living objects using the Pearson's correlation coefficient was the most efficient approach for feature selection. In the other hand, for this problem (8-Class Knn Classification) visualising the dataset using strip plot showing each feature's distribution across the 8 classes for a visual assessment of the features and select the most discriminative.



As the plot clearly shows, the features with the least overlap across the 8 classes are aspect_ratio, hollowness, eyes and cols_with_2. each feature tends to separate a distinct subset of classes, meaning their overlapping regions are largely different from one another making them complementary

Section 2.1

After fitting the KNN classifier with the whole dataset, test it using the same data set and calculate the accuracy of the model all inside a loop iterating the value of K with odd numbers between 1-15 inclusive.

when the value of K was 1 the model achieved an accuracy of 0.9911 showing that the bias at its lowest and the variance at its highest. In this case using, This due to the reason that the nearest neighbour to the predicted value is itself but if other data was used for testing, the model will fail to predict at this level of accuracy because it is overfitted and very sensitive to noise. As the value of K increased the accuracy steadily decreases from 0.9911 to 0.8125, reflecting the bias-variance trade off.

```
-----
Task 2.1: KNN model: K value/accuracy
-----
k= 1 Accuracy: 0.9911
k= 3 Accuracy: 0.9375
k= 5 Accuracy: 0.8929
k= 7 Accuracy: 0.8839
k= 9 Accuracy: 0.8393
k=11 Accuracy: 0.8125
k=13 Accuracy: 0.8125
k=15 Accuracy: 0.8125
```

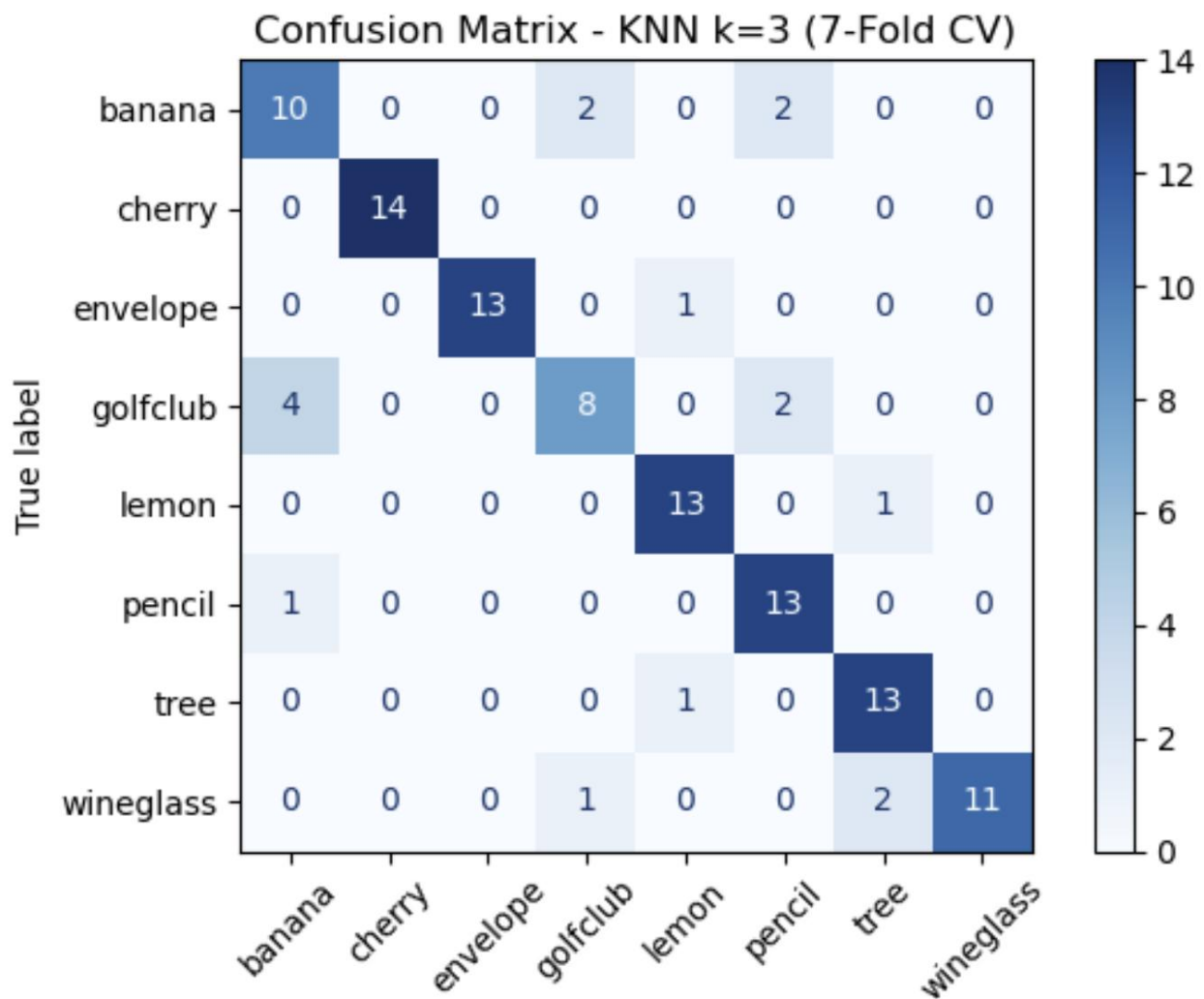
Section 2.2

A 7-fold CV was performed on the KNN model with odd K values between 1 and 15 inclusive, using the same features as Task 2.1. The resulting average CV accuracies were lower than the resubstitution accuracies from Task 2.1 across all K values, which is expected since the models were evaluated on unseen data rather than the data they were trained on. Even if Task 2.1 had used a holdout approach instead, CV would still be expected to produce a more reliable performance estimate as it tests every sample exactly once across all folds, reducing dependence on any single random split. The optimal K value is 3, consistent with Task 2.1, but with a considerable accuracy difference of 0.0893 between the resubstitution accuracy (0.9375) and the CV accuracy (0.8482), highlighting the extent of overfitting in the resubstitution approach. The CV accuracy is the more trustworthy estimate of true generalisation performance.

```
=====
Task 2.2: KNN with 7-Fold CV
=====
k= 1 CV Accuracy: 0.8839
k= 3 CV Accuracy: 0.8482
k= 5 CV Accuracy: 0.8214
k= 7 CV Accuracy: 0.8214
k= 9 CV Accuracy: 0.8125
k=11 CV Accuracy: 0.7946
k=13 CV Accuracy: 0.8036
k=15 CV Accuracy: 0.8036
```

Section 2.3

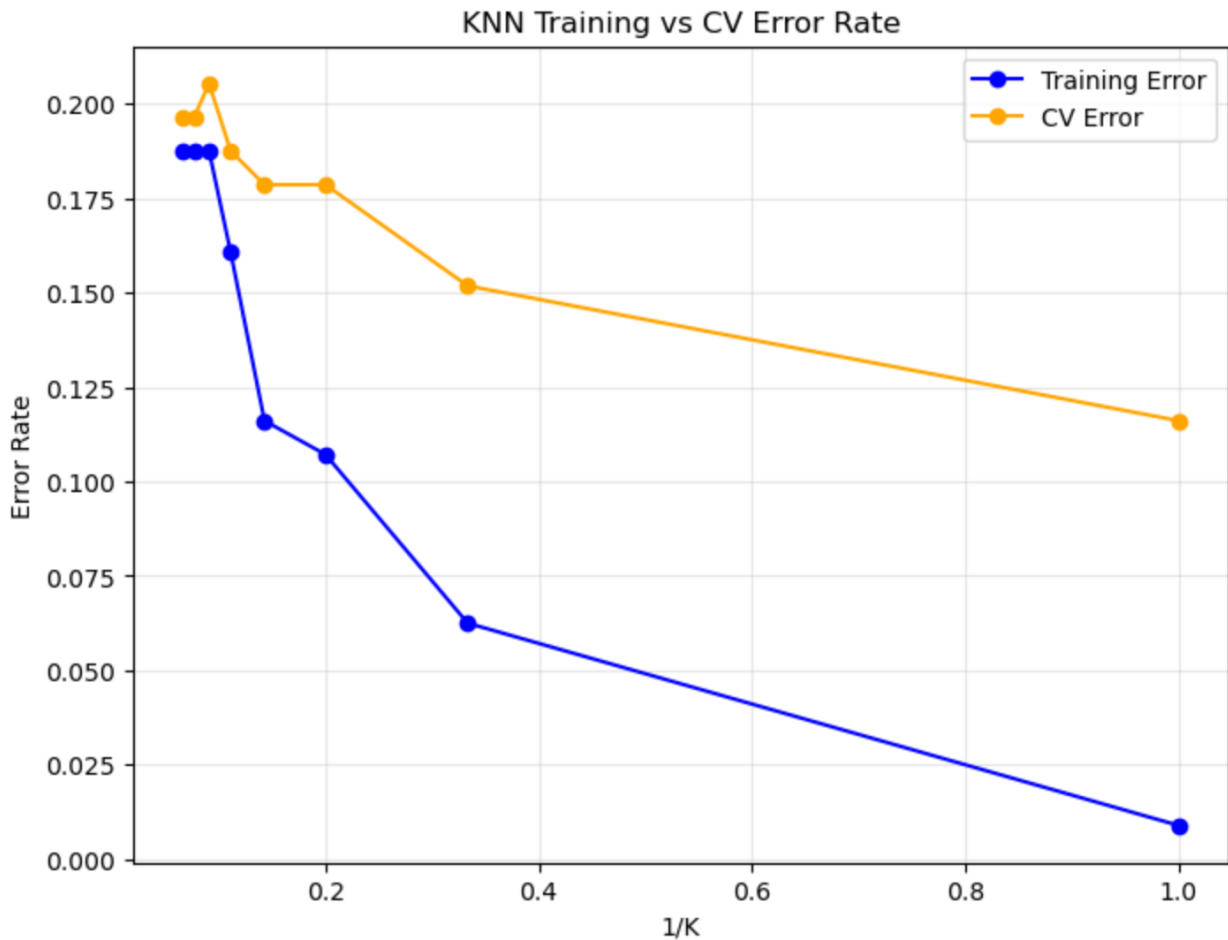
Using $K=3$ as the best value as shown in the previous task, has resulted in a good predictions generally identifying all cherry images and 13 out of 14 of tree, pencil, lemon, lemon and envelope images. However, the model had low accuracy to identify some objects with worst performance on identifying golfclub only 8 /14 true positives and banana with only 10/14. 4 golfclub images were predicted as banana and 2 banana as golfclub suggest a real overlapping between the two classes. Looking to the strip plots(in task 2 introduction) of the selected features, the eyes shows almost identical distribution between the two classes, providing a little discriminative power. The aspect_ratio feature shows a clear overlapping, although their distribution is not identical. While the hollowness and cols_with_2 show less overlap between those two classes, they are insufficient to fully compensate for the similarity in eyes and aspect_ratio. This overlapping makes it difficult for the model to distinguish between these classes as the distance between their points is smaller than the distance to their own class points in those cases.



Section 2.4

To plot the Error rate against $1/k$ for both training and CV Errors, a calculation of the error rate from the accuracy rate was performed for the two models as well as for the inverse of the K value. Then using mat plot library the diagram was plotted. As the plot shows, the error rate was at its highest for both curves when $1/K$ is less than 0.3 (when K is larger than 3). The CV's error rate reach more than 0.2 and the training error rate reach more than 0.18 which suggest a high bias and underfitting. As the value of $1/K$ increase the error rate decrease reaching near 0 for the training and less than 0.125 for the CV when $1/k=1$ and $k=1$. This

suggest a high variance and clear overfit. The balance between variance and bias is shown to be between $1/k = 0.3$ and 0.4 making 3 the optimal value of K .



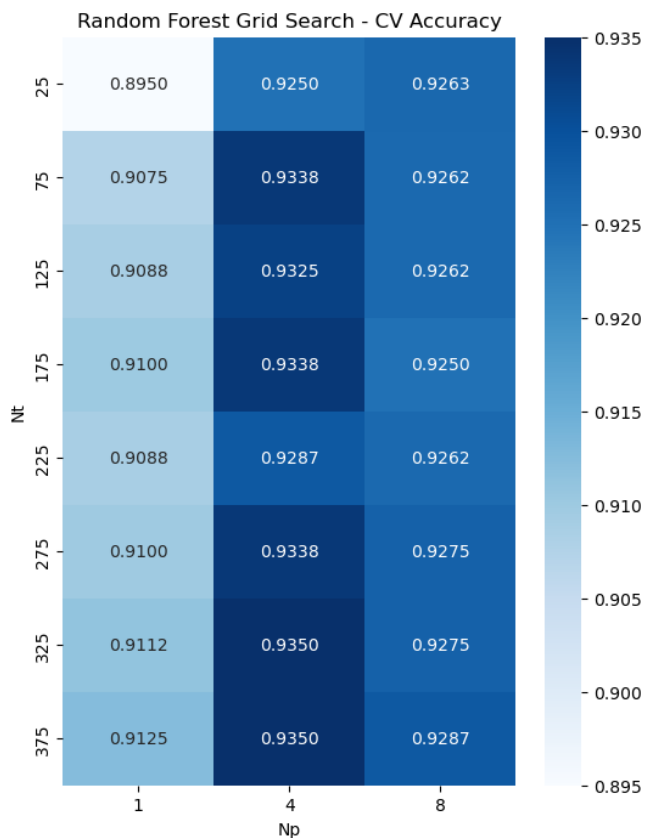
Section 3

For this Section, Train Data Set 3 and Test Data Set 2 were used.

Section 3.1

A 5-fold cross validation random forest classification was preformed using a number of trees between 25 and 375 in increments of 50 and number of predictors of 1, 4 and 8 to find the combination of N_t and N_p that gives us the highest accuracy rate. To achieve this, SKlearn's StratifiedKFold method was used to split data for the 5 folds and RandomForestClassifier to compute the model.

The results (as shown in the figure below) shows a general positive correlation between the number of trees (N_t) and the accuracy especially when the predictors number is 1 or 4. On the other hand, the N_p resulted in the best values when its 4. In the contrary of the initial expectation that one predictor model will not be a competitor to a multi-predictor model, the results show that only a 0.0225 difference between the best multi-predictor model accuracy and single one. This difference can be insignificant in a limited-resource situation. Finally, the best hyperparameter combination to result in the highest accuracy is $N_t = 325$ and $N_p = 4$. While $N_t = 375$ with the same N_p gave the same accuracy rate this was chosen because its similar model. The results show other combinations for simpler models which can be selected depending on the computational limitations and the significance of the accuracy rate.



Section 3.2

To assess the variability of the best model (Nt=325, Np=4), it was refitted 12 times using different random seeds. The CV accuracies ranged from 0.9313 to 0.9388, with a mean of 0.9341 and a standard deviation of 0.0020, indicating that the model is highly stable.

A one-sample t-test was conducted to determine whether the model performs significantly better than random chance. Since there are 8 classes, the chance level is $1/8 = 0.125$. The t-statistic of 1399.10 and a p-value of essentially 0 ($p < 0.05$) provide evidence that the model performs significantly better than chance.

```
=====
Task 3.2: Random Forest Stability Check
=====
Run 1 CV Accuracy: 0.9388
Run 2 CV Accuracy: 0.9338
Run 3 CV Accuracy: 0.9338
Run 4 CV Accuracy: 0.9338
Run 5 CV Accuracy: 0.9350
Run 6 CV Accuracy: 0.9313
Run 7 CV Accuracy: 0.9313
Run 8 CV Accuracy: 0.9350
Run 9 CV Accuracy: 0.9350
Run 10 CV Accuracy: 0.9350
Run 11 CV Accuracy: 0.9325
Run 12 CV Accuracy: 0.9338
```

Mean Accuracy : 0.9341

Std Deviation : 0.0020

t-statistic : 1399.1034

p-value : 0.000000

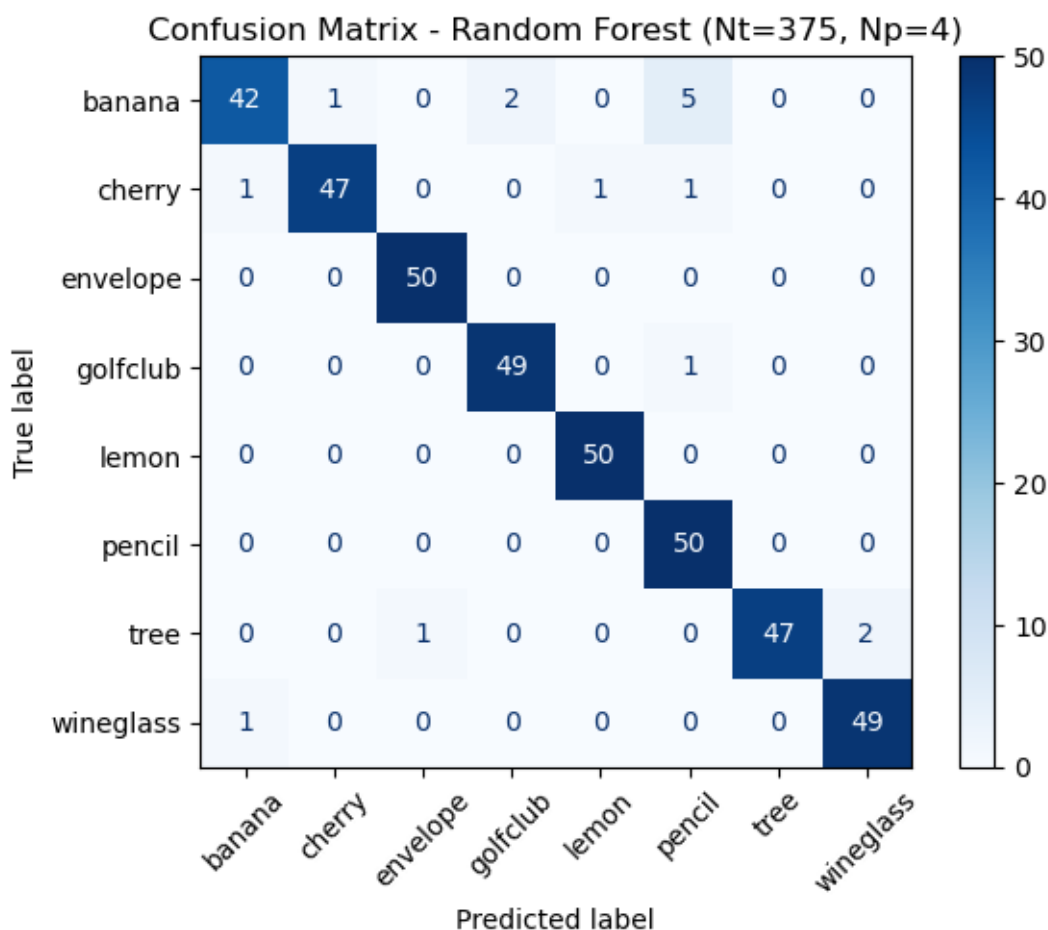
Result: Model performs significantly better than chance ($p < 0.05$)

Section 3.3

The model (Nt=325, Np=4) was trained on the full 800 item training set and tested on all 400 item test set, achieving an accuracy of 0.96. this is higher than the cv accuracy of 0.9341 in task 3.2, which is expected since this model was trained with the sull training set. As per class metrics, envelope, lemon and pencil achieved 1.0 recall identifying every item of these classes and a precision of 1.0 for tree with no false positive misclassification for this class. The worst performance was identifying the banana with a recall of 0.84 and misclass them as a pencil for 5 items. This is the main reason that the pencil class precision is the lowest with 0.88. this misclassification might be to the overlapping between those two classes features.

```
=====
Task 3.3: Final Model Evaluation on Test Set
=====
Test Accuracy: 0.9600
```

	precision	recall	f1-score
banana	0.95	0.84	0.89
cherry	0.98	0.94	0.96
envelope	0.98	1.00	0.99
golfclub	0.96	0.98	0.97
lemon	0.98	1.00	0.99
pencil	0.88	1.00	0.93
tree	1.00	0.94	0.97
wineglass	0.96	0.98	0.97



Conclusions

A Variety of approaches was used for feature selection, classification and model evaluation throughout the project showing a deep understanding of the module's concepts.

References

- [1] Python Software Foundation. Python Language Reference, version 3.x. Available at: <https://www.python.org>
- [2] Harris, C.R. et al. (2020). Array programming with NumPy. Nature, 585, 357-362. Available at: <https://numpy.org>
- [3] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference. Available at: <https://pandas.pydata.org>
- [4] Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95. Available at: <https://matplotlib.org>
- [5] Waskom, M. (2021). Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021. Available at: <https://seaborn.pydata.org>
- [6] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. Available at: <https://scikit-learn.org>
- [7] Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods, 17, 261-272. Available at: <https://scipy.org>